Kryptoanaliza stosowana 2026

Lista zadań nr 1: kryptoanaliza szyfrów klasycznych

Na zajęcia 12 października 2026

Zadanie 1 (2 pkt). Rozważmy szyfr, w którym zbiory tekstów jawnych $\mathcal{P}$ i szyfrogramów $\mathcal{C}$ mają tyle samo elementów ($|\mathcal{P}| = |\mathcal{C}|$). Wówczas dla każdego klucza $k \in \mathcal{K}$ ze zbioru kluczy $\mathcal{K}$ szyfrowanie $\mathrm{enc}_k : \mathcal{P} \to \mathcal{C}$ jest bijekcją (czemu?). Przypuśćmy, że wśród tekstów jawnych jest pewien podzbiór tekstów „sensownych" $\mathcal{S} \subseteq \mathcal{P}$. Ustalmy sensowny tekst jawny $p \in \mathcal{S}$ i klucz $k \in \mathcal{K}$. Taki klucz $k' \neq k$, że $\mathrm{dec}_{k'}(\mathrm{enc}_k(p)) \in \mathcal{S}$ nazywamy *kluczem rzekomym* (*spurious key*). Niech $r(p,k)$ będzie zbiorem kluczy rzekomych dla tekstu jawnego p i klucza k i niech $n(p,k) = |r(p,k)|$. Np. jeśli rozważymy szyfr Cezara na 5-literowych słowach angielskich, to $r(\texttt{RIVER}, \texttt{E}) = \{\texttt{V}\}$, gdyż $\mathrm{dec}_{\texttt{V}}(\mathrm{enc}_{\texttt{E}}(\texttt{RIVER})) = \texttt{ARENA}$ jest sensownym słowem i nie ma innych kluczy poza `E` i `V`, które dawałyby sensowny tekst jawny dla szyfrogramu $\texttt{VMZIV} = \mathrm{enc}_{\texttt{E}}(\texttt{RIVER})$. Atak siłowy ze znanym jedynie szyfrogramem (*ciphertext-only attack*) daje jednoznaczne rozwiązanie szyfrogramu, jeśli $n(p,k) = 0$.

Niech P i K będą niezależnymi zmiennymi losowymi przyjmującymi wartości, odpowiednio, w zbiorach $\mathcal{P}$ i $\mathcal{K}$ oraz niech $C = \mathrm{enc}_K(P)$. *Model szyfru losowego* (*random cipher model*) zakłada, że rozkład zmiennej losowej $\mathrm{dec}_K(C)$ jest jednostajny na zbiorze $\mathcal{P}$. Zauważ, że z tego założenia wynika, że wartość oczekiwana

$$E(n(P,K)) \quad = \quad \frac{|\mathcal{S}|}{|\mathcal{P}|} \cdot |\mathcal{K}| - 1.$$

Będziemy dalej zakładać, że rozważane w tym zadaniu szyfry są zgodne z modelem szyfru losowego.[1]

Niech dany będzie alfabet $\mathbb{A}$ i niech $\mathcal{P} = \mathcal{C} = \mathbb{A}^m$ dla ustalonego m. *Długością krytyczną* (*unicity distance*) nazywamy najmniejszą taką liczbę m, że dla losowo wybranych wartości $p \in \mathcal{P}$ i $k \in \mathcal{K}$ mamy $n(p,k) = 0$ z dostatecznie dużym prawdopodobieństwem. Okazuje się, że dobrym oszacowaniem jest najmniejsza wartość m, dla której $E(n(P,K)) < 1$ (to jest bardzo intuicyjne, ale nie chcemy zbyt głęboko wchodzić w rozważania probabilistyczne, więc po prostu przyjmijmy, że tak jest).

Niech $\mathbb{A}$ będzie 26-literowym alfabetem angielskim. Entropia metryczna[2] języka angielskiego wynosi około 1.2 bita na znak. Oznacza to, że liczba sensownych tekstów angielskich długości m jest rzędu $2^{1.2 \cdot m}$ (podczas gdy wszystkich tekstów jest oczywiście 26^m). Jaka jest *unicity distance* dla a) szyfru Cezara, b) monoalfabetycznego szyfru podstawieniowego, c) k-kolumnowego szyfru przestawieniowego, d) szyfru Vigenère'a, e) *one-time pada*?

W standardowym kodowaniu US-ASCII znakom alfabetu są przyporządkowane ciągi 7-bitowe. Najbardziej znaczący, ósmy bit jest przeznaczony do przechowywania sumy kontrolnej.[3] Współcześnie często bit parzystości nie jest wykorzystywany i wówczas zawsze ma wartość zero.

Szyfrujemy tekst długości m znaków w kodowaniu US-ASCII za pomocą szyfru komputerowego wykorzystującego klucz o długości k bitów. Wyznacz *unicity distance* w tym przypadku. Wyznacz wartość m dla $k = 56$, 64, 128, 192 i 256 bitów. Opisz algorytm ataku siłowego (*brute force*) na szyfrogram odpowiedniej długości, jeśli w kodowaniu a) ósmy bit jest zawsze zerem, b) ósmy bit jest bitem parzystości.

W pewnym formacie zapisu dane (które mogą być dowolne) są poprzedzone pewną liczbą bajtów zawierających zera. Używamy szyfru komputerowego wykorzystującego klucz o długości k bitów. Ile bajtów zer powinno poprzedzać dane, aby atak siłowy na pojedynczy szyfrogram dał z dużym prawdopodobieństwem jednoznaczne rozwiązanie?

Pytanie filozoficzne: czy opisane wyżej ataki, to ataki tylko z szyfrogramem (*ciphertext only*), czy ze znanym tekstem jawnym (*known plaintext*), tj. co to znaczy „znać" tekst jawny? Jak często w praktyce informatycznej mamy na tyle dużo ogólnej wiedzy na temat tekstów jawnych, by *unicity distance* była mniejsza niż rozmiar przechwyconego szyfrogramu?

Zauważ, że entropia dowolnych tekstów ASCII wynosi 7 bitów na bajt, lecz jeśli wiemy, że są to teksty w języku angielskim, to spada do 1.2 bita na bajt. Im większa entropia testów jawnych, tym większa *unicity distance*. Entropię metryczną szyfrowanych danych można zwiększyć przez kompresję. Dlaczego kompresja nie zmieni jednak *unicity distance* dla tekstu jawnego?

[1] Tj. zakładamy, że odpowiedni rozkład jest bliski jednostajnemu, a powyższa równość zachodzi w przybliżeniu. Zob. Alfred J. Menezes, Paul C. Van Oorschot, and Scott A. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1996, podrozdział 7.3.5, str. 245–250 oraz Douglas R. Stinson, Maura B. Paterson, *Cryptography. Theory and Practice*, Fourth Edition, CRC Press, 2019, podrozdział 3.5, str. 75–79.

[2] Entropia przypadająca na jeden znak.

[3] Dobiera się go tak, by liczba zapalonych bitów zawsze była parzysta. W szczególności kod znaku `DEL` składa się z ośmiu jedynek.

Zadanie 2 (3 pkt.). Poniższe dwa szyfrogramy, to wynik zaszyfrowania *one-time padem* dwóch tekstów w języku polskim z użyciem tego samego klucza w następujący sposób:

```
alphabet = "AĄBCĆDEĘFGHIJKLŁMNŃOÓPQRSŚTUVWXYZŻŹ"
alphasize = len(alphabet)
num2char = dict(enumerate(alphabet))
char2num = { num2char[n]:n for n in num2char }
def encodeChar(c, k):
    return num2char[(char2num[c]+char2num[k]) % alphasize]
def encode(plaintext, key):
    return ''.join(map(encodeChar, plaintext, key))
```

Spacje i znaki przestankowe usunięto z tekstu.

ĘĘKGCATDUJXNYYVXWÓWUĄVŹGJĘCŃŁQŃJAŁGEJWXNYĆŃKXŁMJMOWŃAVVCUBWLŃARJHÓŁVBIONKFQJMEĄŃ
ŹJAEHĆQEŁRIŁŚŹFMŹNŚŁFAĘXTOPŚŻUĆŃPBUŚAGWYZGHUHRWUGKFŚKĄYV

ŁUXBUQLĄNUAÓĘCŃŁĆŚDNŁVSSGĘGKĆPLIRDWVÓDŃŚŁÓĘŃNWWOŚZFĆŚSŹBŻUSIKUĆHHUĆUĘŁVILCWRŹBAY
MYBKŹŁFRKŃŃCIŹUŻDSTHEOOLLOÓIŁZVPJŁŻWGDŃĆFŹFQKOAĄSZAVĆŻŁŚJADLCŹDWEŁHBĆQZXPĘINLĘYX
YAŻTLQMNĄĆŻWU

Odszyfruj te wiadomości. Przygotuj krótką prezentację, jak przebiegała kryptoanaliza i z czym były problemy.

Zadanie 3 (1 pkt.). Napisz odpowiednie skrypty i przygotuj krótką prezentację następującego protokołu, który ma zapewnić *plausible deniability*.

Adam i Bob działają w podziemiu. W ważnym momencie, kiedy nie będą razem, Adam musi przesłać Bobowi bardzo tajną wiadomość, której ujawnienie narazi ich komórkę ruchu oporu na wielkie niebezpieczeństwo. Ustalają więc szczegóły szyfrowania tej wiadomości. Szyfrogram może jednak zostać podsłuchany przez władze, które wówczas ich aresztują i zmuszą torturami do ujawnienia klucza. Adam i Bob umawiają się więc na następujący protokół.

- Generują losowo i zapamiętują klucz szyfrujący do *one-time pada*.
- Wybierają fałszywą wiadomość, którą w razie zatrzymania obaj ujawnią jako tę, która była zaszyfrowana. Wiadomość powinna być dostatecznie krępująca, żeby uwiarygodnić, dlaczego zdecydowali się na jej zaszyfrowanie (nie może więc też być ewidentnie fałszywa), ale nie na tyle inkryminująca, żeby ponieśli poważne konsekwencje jej ujawnienia.
- Gdy trzeba, Adam szyfruje tajną wiadomość umówionym kluczem i wysyła ją do Boba, po czym zapomina prawdziwy klucz. Bob odszyfrowuje wiadomość i również zapomina prawdziwy klucz.
- Obaj wyliczają nowy, fałszywy klucz, tak dobrany, żeby przechwycony przez władze szyfrogram rozszyfrowywał się do umówionej wcześniej fałszywej wiadomości.
- W razie zatrzymania obaj (dostatecznie niechętnie) ujawniają władzom ten sam fałszywy klucz i tę samą, fałszywą wiadomość.

Zadanie 4 (1 pkt.). W tym łatwym zadaniu będziemy się bawić szyfrem Vigenère'a i jego szczególnym przypadkiem (gdy klucz jest jednoliterowy) — szyfrem Cezara.[4] Zagadki kryptograficzne w tym zadaniu zaszyfrowałem używając następującego kodu w Pythonie:

```
from itertools import cycle
alphabet = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
alphalen = len(alphabet)
num2char = dict(enumerate(alphabet))
char2num = { num2char[n]:n for n in num2char }
def encodeChar(c, k):
    return num2char[(char2num[c]+char2num[k]) % alphalen]
def encode(plaintext, key):
    return ''.join(map(encodeChar, plaintext, cycle(key)))
```

Alfabet (zarówno tekstu jawnego, jak i szyfrogramu), jest 26-literowym alfabetem języka angielskiego. Nie zawiera spacji, którą wraz ze znakami przestankowymi pomijamy.

[4] Zauważ, że drugim szczególnym przypadkiem szyfru Vigenère'a jest *one-time pad* — gdy tekst jawny jest nie dłuższy niż klucz.

1. Następujący szyfrogram został zaszyfrowany szyfrem Cezara:

   ```
   FDGEFYQUMYMODKBFASDMBTQD
   ```

   Złam go: podaj klucz i tekst jawny.

2. Następujący szyfrogram został zaszyfrowany szyfrem Vigenère'a:

   ```
   XHQPMFTFSJBHAMEHGIGHISHLPHLJAECWRVSRJWXNQECBSI
   QSCQSRHERWTWSVLVMRVLJAECWRVSRJWXNQECBSIFIHCPKS
   ```

   Złam go wiedząc, że tekst jawny zaczyna się słowami `ITMAY` (tzn. „It may" po przepisaniu ze spacjami i małymi literami).

Zadanie 5 (1 pkt.). Rozważmy teraz alfabet języka polskiego:

```
alphabet = "AĄBCĆDEĘFGHIJKLŁMNŃOÓPQRSŚTUVWXYZŻŹ"
```

Następujący szyfrogram został zaszyfrowany szyfrem Vigenère'a, podobnie jak w Zadaniu 4:

```
ŃABQYĆVLHĄOHQKĆNVODGQVŹŁFIXDWUJOZQNXŃNMRHZŹVSUJDFCWUFKRYĘSUÓLIWZEĆFHĄZNŚĘRKN
YRZXCSIKQKĘVZZSHAMĆULNBSHJTÓHINQDOWŹKŁGSDATFŹCJDNMYAABCWXQOŻKĘYXTCXŹGZQMĆBZI
ĘZEXSŃLĄFŹPDVNYNVŹZCIOFSQŻZKGAPXÓŁRUŃŃIQRUÓŁNZSYHQPBLIMŻAOPĄWLNBCEŚXSZĄÓOCUA
ZXTXCUJUĆIHĄNAGGIUNMBCBDXRQLOBIQOBYPIPZXXĘZĘONOTŹQIULUXHRNAKSHBŹĘTFXWZSĘĘÓBV
IBJRAÓĘWWQŹZEĆDTNŹNŚ
```

Złam go wiedząc, że tekst jawny zawiera słowo `KULTURY`.

Zadanie 6 (2 pkt.). W tym zadaniu rozważamy szyfr podstawieniowy monoalfabetyczny. Kluczem jest napis zawierający permutację liter alfabetu, np.

```
key = 'GMLXVKQT*PFRBUOSJHNYZAIEDWC'
```

W powyższym przykładzie alfabet (zarówno tekstu jawnego, jak i szyfrogramu) składa się z 26 liter języka angielskiego oraz gwiazdki, która reprezentuje spację:

```
alphabet = '*ABCDEFGHIJKLMNOPQRSTUVWXYZ'
```

Do zaszyfrowania tekstu użyłem następującego kodu w Pythonie:

```
def encode(plaintext, key):
    alphabet = list(key)
    alphabet.sort()
    encdic = dict(zip(alphabet, key))
    return ''.join(map(lambda c: encdic[c], plaintext))
```

Złam poniższy szyfrogram wiedząc, że tekst jawny jest w języku angielskim, a alfabetem jest, jak powyżej, zbiór złożony z 26 liter oraz gwiazdki.

```
LPMUQEMQMGKLWDPMCYRHMHQXMLBMQSKLRMQBHMPDVMCRYCJEMUVKVMEPKLJLBWMPDLKPVVBMULBEPYBM
EILPDMDLEMCDLBMBAZZRVHMLBPYMDLEMGKVQEPMLBMQBMVOOYKPMPYMVECQSVMPDVM*LRVMULBHMERLS
SVHMTALCJRXMPDKYAWDMPDVMWRQEEMHYYKEMYOM*LCPYKXMIQBELYBEMPDYAWDMBYPMTALCJRXMVBYAW
DMPYMSKV*VBPMQMEULKRMYOMWKLPPXMHAEPMOKYIMVBPVKLBWMQRYBWMULPDMDLIMPDVMDQRRUQXMEIV
RPMYOMGYLRVHMCQGGQWVMQBHMYRHMKQWMIQPEMQPMYBVMVBHMYOMLPMQMCYRYAKVHMSYEPVKMPYYMRQK
WVMOYKMLBHYYKMHLESRQXMDQHMGVVBMPQCJVHMPYMPDVMUQRRMLPMHVSLCPVHMELISRXMQBMVBYKIYAE
MOQCVMIYKVMPDQBMQMIVPKVMULHVMPDVMOQCVMYOMQMIQBMYOMQGYAPMOYKPXMOL*VMULPDMQMDVQ*XM
GRQCJMIYAEPQCDVMQBHMKAWWVHRXMDQBHEYIVMOVQPAKVEMULBEPYBMIQHVMOYKMPDVMEPQLKEMLPMUQ
EMBYMAEVMPKXLBWMPDVMRLOPMV*VBMQPMPDVMGVEPMYOMPLIVEMLPMUQEMEVRHYIMUYKJLBWMQBHMQPM
SKVEVBPMPDVMVRVCPKLCMCAKKVBPMUQEMCAPMYOOMHAKLBWMHQXRLWDPMDYAKEMLPMUQEMSQKPMYOMPD
VMVCYBYIXMHKL*VMLBMSKVSQKQPLYBMOYKMDQPVMUVVJMPDVMORQPMUQEMEV*VBMORLWDPEMASMQBHMU
LBEPYBMUDYMUQEMPDLKPXMBLBVMQBHMDQHMQM*QKLCYEVMARCVKMQGY*VMDLEMKLWDPMQBJRVMUVBPME
RYURXMKVEPLBWMEV*VKQRMPLIVEMYBMPDVMUQXMYBMVQCDMRQBHLBWMYSSYELPVMPDVMRLOPMEDQOPMP
DVMSYEPVKMULPDMPDVMVBYKIYAEMOQCVMWQZVHMOKYIMPDVMUQRRMLPMUQEMYBVMYOMPDYEVMSLCPAKV
EMUDLCDMQKVMEYMCYBPKL*VHMPDQPMPDVMVXVEMOYRRYUMXYAMQGYAPMUDVBMXYAMIY*VMGLWMGKYPDV
KMLEMUQPCDLBWMXYAMPDVMCQSPLYBMGVBVQPDMLPMKQBM
```

Zauważ, że pozostawienie separatora słów znacznie ułatwia kryptoanalizę.

Zadanie 7 (2 pkt.). Oto szyfrogram tekstu w języku angielskim zaszyfrowany szyfrem podstawieniowym. Alfabetem jest zbiór 26 liter angielskich. Spacje pominięto.

```
KMFXLZMKDZVOXLZPXKYIPBKRRZFSVYDSVVFKMFXLZXLIYFAKYXVOXLZSKXZYPQZBKDZSVYDSVVFKMFDK
MCDZMFIZFVOXLZSKXZYPQZBKNPZXLZCSZYZDKFZQIXXZY
```

Odszyfruj ten tekst wiedząc, że występuje w nim słowo `WATERS`. Przygotuj krótką prezentację, jak przebiegała kryptoanaliza i z czym były problemy.

Zadanie 8 (1 pkt.). Przygotuj krótką prezentację metod kryptoanalizy szyfru Vigenère'a: testu Kasiskiego i indeksu koincydencji Friedmana.[5] Napisz odpowiednie skrypty i przygotuj przykłady.

Zadanie 9 (1 pkt. za każdy szyfrogram). Oto trzy teksty zaszyfrowane szyfrem podstawieniowym, pochodzące z książki Helen Fouché Gaines, *Elementary Cryptanalysis*, 1939. Tradycyjnie litery są grupowane po pięć dla łatwiejszego przepisywania. Złam te szyfry.

```
SCYJT OPNRM JTUEA WSROR OAEPQ RJCRO ARMPH
QKJQS RSJHA XPFKE AQRMY SRPQP MPSEC AHGAW
SROPE EESHA QOPVS HIROA QPFAE AHIRO PHNPQ
RJHTF UAMCJ MRYRO MAAWA EEBTQ RWMSR ASRJH
AIMJT KUAEJ WPHJR OAMPH NQAAW OPRYJ TQAAL.
```

```
JCWEH SNDFS BNJIV TEAGV DHOCQ QIQFR PHFKQ
EARFQ ARFAH FQEJC BNJNH BEOCB NLNOV HBLFQ
JBNAB LFVHC AJIVB NWNST BLEAG VAJNS RFWNS
YRVSS CAEHV AQFCJ EAGJN AWNSO VBVCQ YDCSP
HEHOC SPEAG BEONA FRLCA GNEAK CSONS HACBE
FACQX.
```

```
BOYBA NKILL APKRI YAPYY UPBLY ERPBP LGYGM
HLABO YKJAK LPYLH HJACR PORCQ UYNBH LABOY
GNAZN YLHBO YKNAN PRBRW OJCBR CQDNP K.
```

[5] Zob. np. Douglas R. Stinson, Maura B. Paterson, *Cryptography. Theory and Practice*, Fourth Edition, CRC Press, 2019, podrozdział 2.2.3, str. 45–48.